

**FEDERAL UNIVERSITY OF RIO DE JANEIRO
POLYTECHNIC SCHOOL
DEPARTMENT OF ELECTRONIC AND COMPUTER
ENGINEERING**

UNDERGRADUATE PROJECT PROPOSAL

Student: Matheus Hoffmann Fernandes Santos
hoffmann@poli.ufrj.br

Advisor: To be defined, Dr.

Co-advisor: Pedro Veras Guimarães, Dr. (LabECO, Department of Mechanical
Engineering, UFSC)

1. TITLE

Operational optimisation of the WAVEWATCH III® wave model in the ReNOMO forecast cycle (LabECO/UFSC): build options, run configuration, modern Fortran and rewriting kernels in C++/Kokkos for GPU.

2. EMPHASIS

Computing / High-performance scientific computing.

3. THEME

Third-generation spectral wave models solve the wave action balance equation on a spatial grid and a spectral grid of frequencies and directions, integrating wind input, nonlinear interactions between components and dissipation at every point and every time step, with no *ad hoc* assumption about the shape of the spectrum (The WAMDI Group, 1988; Tolman, 1991). WAVEWATCH III® (WW3), developed at NOAA/NCEP, is the reference model of this class, maintained by NOAA/NCEP and distributed as open source for operational and research use (NOAA-EMC, 2026; The WAVEWATCH III Development Group, 2019). It is written in Fortran, parallelised with MPI and OpenMP, and a *switch file* fixes at compile time which parts of the code enter the executable: the physics, the propagation scheme and the I/O options (The WAVEWATCH III Development Group, 2019).

Wave forecasting inherits the nonlinearity of the atmospheric forecast that drives it, and its skill degrades with lead time: the assessment of the NCEP ensemble system against four altimeter missions shows the error growing along the forecast

days, with the ensemble mean cutting the scatter error of significant wave height from 1.8 m in the control run to 1.3 m (Campos et al., 2020). The operational answer is to run more often and in ensemble, perturbing initial conditions and forcing (Farina, 2002). NCEP, for instance, runs GFS-Wave four times a day (00, 06, 12 and 18 UTC) out to 384 h on global grids of 0.16° and 0.25° with regional nests, and GEFS-Wave with 30 members at 0.25° out to 16 days (NOAA/NWS/NCEP/EMC, 2026). Every one of those choices — cycle frequency, resolution, number of nested domains and of members — is bounded by the wall-clock time of a forecast and by the computing resources available: wall-clock time decides how much physics and how much resolution fit inside the delivery deadline.

The Laboratório de Engenharia e Ciências Oceânicas (LabECO) of the Department of Mechanical Engineering at the Universidade Federal de Santa Catarina (UFSC) runs WW3 as part of its contribution to the Rede Nacional de Observação e Monitoramento Oceânico (ReNOMO), a national network funded by the CNPq/MCTI/Finep 062/2022 call that integrates ocean observation, quality control and operational product delivery for the South Atlantic and the Brazilian coast (LabECO/UFSC, 2026; UFSC, 2026). That chain, rather than a laboratory installation, constitutes the reference case of this work.

The theme of this project is the **parallel simulation and GPU acceleration of a third-generation spectral wave model**, taking WW3 in the configuration operated by LabECO as its case. It is a performance-engineering problem with a measurable result — the wall-clock time of a forecast cycle — under the constraint that the correctness of the forecast fields is never traded for speed.

4. SCOPE

The work is limited to the single-grid driver of WW3 (`ww3_shel`), with the test cases coming from the operational forecasting system of LabECO (UFSC), which is made available to the project at the start of the schedule: its grid, its switch file, its forcing and its output list. That configuration and the exact source revision are frozen and documented at the start of the project, so that every measurement refers to the same case. The regular grid is the reference case; unstructured grids (PDLIB) enter the scope only if the laboratory’s operational configuration requires them, and in that case the same measurement procedure applies to them.

Four rungs are in scope, attempted in order and each gated by a comparison against the frozen reference run:

1. **Build options.** Compiler, optimisation flags, the switch file (in particular the choice between shared-memory, MPI and hybrid builds, and the NetCDF-4 output path), and the MPI and OpenMP layout. This rung changes no source code and is expected to be bit-for-bit reproducible against the reference or, when floating-point reordering is involved, within round-off.
2. **Run configuration.** Domain decomposition, time steps allowed by the CFL condition, output frequency and output fields, restart strategy, and forcing interpolation. This rung changes namelist inputs only.
3. **Targeted modern-Fortran refactoring, LLM-assisted.** Rewriting of the routines that the profile identifies as dominant, typically the source-term integration (`W3SRCE`) and spatial propagation, using standard Fortran 2008/2018 constructs (`do concurrent`, explicit interfaces, contiguous arrays) without altering the numerical algorithm and within a tolerance defined per output field. The migration is carried out with an LLM-based coding assistant directed by the student, in the role Koldunov et al. (2026) describe for FESOM2: the model proposes the translation, routine by routine, while the student and the co-advisor own the review and the validation. What enters the repository is the evidence, not the suggestion: prompts, the reviewed diff and the parity test results are versioned with the code, and no routine is accepted without passing the per-field comparator described in the methodology.
4. **Rewriting kernels in C++/Kokkos and running them on a GPU (NVIDIA H100).** Kokkos is a C++ performance-portability library: the same source, written once with its parallel abstractions (`parallel_for`, `parallel_reduce`) and data abstractions (`View`, which fixes the memory layout per architecture), is compiled to the native backend of each machine — Serial and OpenMP on CPUs, CUDA on NVIDIA GPUs, HIP on AMD and SYCL on Intel (Trott et al., 2022). It is the same layer named for WAVEWATCH IV (Tolman, 2026), which makes the artefacts reusable. Substitution is incremental and bottom-up: the routines that remain dominant after rung 3 become Kokkos kernels, called from Fortran through `bind(C)` interfaces and kept in the same binary

as the original routine, behind a run-time switch. It starts with the nonlinear interaction term and follows the profile order. This is not a full port of the model: the state stays in Fortran and the CPU–GPU transfer cost per time step is measured and reported. A GPU kernel reaches the laboratory’s operational configuration if, and only if, it reduces the run time of the operational case and meets the same parity criteria as the earlier rungs.

The order of the rungs is part of the scope: rungs 1 and 2 are exhausted before any code change, and rung 3 before rung 4, so that the C++/Kokkos rewrite is only attempted on routines whose residual cost has already been demonstrated.

There is currently an initiative to convert the code to C within the WAVEWATCH IV project (NOAA-EMC, 2026). This proposal is conducted independently of that repository and does not intend to contribute to it: it adopts only the WW4 test structure (unit and integration levels in GoogleTest) for the rewritten kernels, so that the artefacts remain useful in a future migration.

Out of scope are the multi-grid driver (`ww3_multi`), coupling to other models, data assimilation, and any change to the physics parameterisations.

5. JUSTIFICATION

An operational forecasting chain is only as useful as its timeliness. For the wave field that feeds the ReNOMO products, a shorter cycle means earlier delivery, room for higher resolution at the same deadline, and the possibility of running several members to quantify uncertainty — the three axes NCEP tunes in its own operational system and which, there as here, are bounded by run time (Campos et al., 2020; NOAA/NWS/NCEP/EMC, 2026). WW3 was designed for the supercomputers of the last two decades and carries choices, in its build system, its data layout and its parallel decomposition, that are not automatically the best ones for today’s machines: multi-core nodes with large memory, fast NVMe storage and, increasingly, GPUs.

The first two rungs are justified by the model’s own documentation: the switch file and the compiler options decide which code is compiled and how, and the parallel layout decides how much time is spent moving spectra between processes rather than computing them (The WAVEWATCH III Development Group, 2019). They are cheap rungs, frequently neglected in installations configured once and never

revisited, and they produce the instrument the later rungs depend on: a reproducible benchmark and a profile. That is why they come first: no rewrite is justifiable before knowing how much of the run time survives them.

The third rung is justified by the structure of the code. Published profiling of WW3 shows that the source-term integration dominates the run time, with propagation second, and that the routines involved use large local arrays and module-level global state, which hinders both vectorisation and any form of offloading (Ikuyajolu et al., 2023). Rewriting them in modern Fortran, preserving the algorithm and the arithmetic, is a well-understood engineering task with directly measurable value in the model the laboratory operates, and it prepares the next rung, because a routine with an explicit interface and contiguous data can be replaced by a kernel.

The fourth rung addresses the question the laboratory actually asks, whether an H100 can accelerate the model in operation. Upstream WW3 has no GPU execution path (NOAA-EMC, 2026). The published port on record moved the source terms to the GPU with OpenACC directives and obtained roughly 1.3 times speed-up per node, limited by host-device traffic at every time step (Ikuyajolu et al., 2023). By contrast, WAM6-GPU, which refactored the whole model to keep the fields resident on the device, reported an order-of-magnitude gain on a multi-GPU node (Yuan et al., 2024). The lesson is that partial directive-based offloading with the state on the host does not pay, and that the gain comes from kernels designed for the device. This project takes the intermediate path demonstrated for FESOM2 by Koldunov et al. (2026), a preprint, who moved some 74 thousand lines of Fortran to C and then to C++/Kokkos in weeks, with an expert-directed LLM coding assistant, two-stage literal translation and a validation ladder. A kernel rewritten this way runs on CPU and on GPU from the same source, because Kokkos compiles that source to each machine’s backend (Trott et al., 2022), and can be validated in isolation against the original Fortran. The criterion for entering operation is the same as for the other rungs, measured gain and no regression, not a decision taken in advance.

State of WW3 testing. The proposed validation rests on the model’s own test infrastructure, surveyed at revision `develop` 7.14 in September 2026 (NOAA-EMC, 2026). WW3 has 62 regression cases in `regtests/`, run by a matrix of scripts that compile switch combinations and compare the outputs of two runs file

by file with `cmp` and `diff`, classifying each case as identical or not identical; the matrix includes specific variants for bit-for-bit reproducibility across restarts, thread counts and process counts. There is no tolerance-based comparison: any numerical difference is reported as a failure and it falls to the reviewer to judge it. Unit tests have existed since 2023 but cover input and output only (five programs via CTest). Public continuous integration compiles with GNU and Intel and runs a single regression case; the full matrix runs on NCEP machines. Recent activity concentrates on cleaning compiler warnings and fixing CI, with intermittent matrix failures open. For this project that defines the method: rungs 1 and 2 are validated with the model’s own matrix, whose bit-for-bit criterion is the right one; rungs 3 and 4 require a per-field comparator with tolerances and per-routine unit tests, which WW3 does not have and which this project builds and delivers.

State of WW4. The WAVEWATCH IV repository was created in November 2025 and, on 15 September 2026, has 36 commits on `develop`, no published release and its last push in August 2026 (NOAA-EMC, 2026). Phase II of the project (languages, governance and architecture) was completed in March 2026 (Tolman, 2026); the current code is a C++ core with utilities and a standalone executable, still without physics. The test strategy is defined at four levels — unit (L1), integration (L2), functional (L3) and regression (L4) — in GoogleTest with CTest, running in CI on every push; L1 and L2 were rewritten in July 2026 to cover all existing code, while L3 and L4, which would replace the WW3 regression matrix, do not yet exist. The build system is being simplified to pure CMake, and the CPU–GPU architecture is an open question, with Kokkos named as the data abstraction layer and the choice between one or two code bases still to be made. The first public release is expected in January 2027, and NOAA’s plan only sunsets WW3 support once WW4 matures (NOAA/NWS/NCEP, 2025). Two consequences follow: WW3 will remain the laboratory’s operational model for years, which justifies optimising it; and the kernels rewritten here, with Kokkos and L1/L2 tests in the WW4 style, can inform the WW4 architecture discussion rather than duplicate it.

6. OBJECTIVE

The general objective is to reduce, in a measured and reproducible way, the run time of the operational WAVEWATCH III cycle maintained by LabECO/UFSC

within ReNOMO, without changing its scientific results beyond the tolerances agreed with the laboratory, exhausting configuration and build optimisations before rewriting any code, and to decide by the same criterion — measured gain and no regression — whether C++/Kokkos kernels running on an NVIDIA H100 GPU enter the operational configuration.

The specific objectives are:

1. To freeze and document the operational configuration (source revision, switches, namelists, grid, forcing, outputs, hardware) and to build a reproducible benchmark of the reference run, with a defined metric (run time per forecast hour).
2. To produce a profile of the reference run by routine and by phase (source terms, propagation, communication, input and output), on one and on several MPI processes.
3. To quantify the gain of the build, configuration and Fortran refactoring rungs, each with its parity evidence against the reference, and to deliver the best configuration to the laboratory as a documented build.
4. To build the validation infrastructure WW3 lacks: a per-field comparator with versioned tolerances and per-routine unit tests on captured inputs, integrated with the model’s regression matrix.
5. To rewrite in C++/Kokkos, in profile order, the kernels that remain dominant, validate them against the original Fortran on CPU, measure their gain on GPU (H100) and decide, on gain and parity, whether they enter the operational configuration.
6. To publish tooling, results and recommendation in the open repository, in a form the laboratory can rerun.

7. METHODOLOGY

The project follows the principle of measuring before optimising: no optimisation is attempted before the reference run is reproducible and profiled, and none is accepted without a parity check against it.

Reference and benchmark. The laboratory’s operational case is captured as a self-contained test: input files, forcing for a fixed period, switch file, namelists and reference results. The same is done with the official regression test closest to the operational configuration. The benchmark script records time per phase, processes

and threads, compiler and options and the source revision, and produces a table that can be attached to the reports.

Profiling. The reference run is profiled with `gprof` or `perf` on one, four and sixteen MPI processes, with communication time recorded separately. The result is a time-per-routine table, grouped into the phases of the time loop, which sets the order of the refactoring and the baseline for every later measurement.

Parity criteria. In rungs 1 and 2 the results are compared bit for bit with WW3’s own regression matrix, used as it is; when a compiler option reorders floating-point operations, the comparison uses per-field tolerances (for example significant wave height, peak period and mean direction) proposed by the student and approved by the co-advisor. For rungs 3 and 4 the project adds what the matrix does not offer: a per-field comparator that reads the NetCDF outputs of two runs and applies tolerances recorded in a versioned file, and per-routine unit tests that run the original and the rewritten routine on inputs captured from the operational case. A change that fails the criterion is not incorporated.

Rungs. Build options are explored first, as a matrix of compiler, options and parallel layout. Run configuration follows, varying one namelist parameter at a time. The Fortran refactoring addresses the routines at the top of the profile, one at a time, in a fork that keeps the original routine reachable through a run-time switch in the same binary. Only then does rung 4 begin, restricted to the routines whose residual cost the updated profile still shows as dominant.

Kernel interoperability and testing. Each C++/Kokkos kernel is exposed to Fortran through an `ISO_C_BINDING/bind(C)` interface, and the original routine stays in the binary behind the same run-time switch, so that the regression matrix and the comparator exercise both paths without recompiling. The layout of the Kokkos `Views` reproduces that of WW3’s spectral arrays, so that the CPU call copies no data. The translation follows the FESOM2 recipe (Koldunov et al., 2026): a literal translation to C, checked against the Fortran on captured inputs; a Kokkos version whose Serial backend must be bit-for-bit identical to the C one; and only then the CUDA backend on the H100, compared statistically, with CPU–GPU traffic per time step recorded. The tests follow the WW4 structure: level L1, a unit test of the kernel on a synthetic (JONSWAP) spectrum with declared and justified

tolerances, in GoogleTest; level L2, a rerun of the closest regression case and of the operational case, comparing WW3 and WW3+Kokkos with the per-field comparator. An LLM-based coding assistant is used for the translation under the same rules, with the student and the co-advisor responsible for validation; prompts and parity evidence are versioned with the code. The decision to take a kernel into operation is made with LabECO on two pieces of evidence: the timing table of the operational case, with and without the kernel, and the parity report.

Tooling and reproducibility. All work is done in the open `ww3-gpu` repository (Hoffmann, 2026), which provides the pinned toolchain, the build and regression scripts, the model as a submodule at a pinned revision, and continuous integration, so that any measurement can be repeated by the laboratory on its own machines. Results are recorded as tables and figures generated by scripts, never by hand.

Deliverables and validation. The deliverables are the configuration record, the benchmark and profile tables, the best documented build, the per-field comparator and the unit tests, the Kokkos kernels with their parity and timing evidence, and the final report with the operational recommendation. Scientific validation rests with the co-advisor at LabECO, who sets the tolerances and reviews the parity evidence; engineering validation rests with the repository’s automated tests and independent reruns at the laboratory.

Risks. The main one is silent divergence of results after a rewrite, mitigated by the parity criteria and the per-routine tests. The second is rung 4 consuming the time of the earlier ones, mitigated by the fixed order of the rungs and by the rule of rewriting only a routine whose residual cost has been measured. The third is the GPU gain being bounded by data traffic while the state remains in Fortran; in that case the result is the measurement of that bound and a recommendation not to run on GPU. The fourth is the operational grid being unstructured; in that case rungs 3 and 4 are restricted to the source terms.

8. SCHEDULE

The project is planned for two academic periods. Dates are proposals to be adjusted with the advisors.

Table 1: Undergraduate project schedule.

Stage	Deadline
Bibliographic review; configuration record of the operational case	10/2026
Reproducible benchmark and profile of the reference run	11/2026
Rung 1: build options (compiler, flags, switch file, parallel layout)	12/2026
Rung 2: run configuration; per-field comparator and per-routine tests	01/2027
Rung 3: targeted modern-Fortran refactoring of the profiled hotspots	02/2027
Rung 4: C++/Kokkos kernels; parity on CPU and measurement on the H100	03/2027
Operational decision with LabECO; final report and revision	04/2027
Defence	from 05/2027

References

CAMPOS, Ricardo Martins *et al.* [Global assessments of the NCEP ensemble forecast system using altimeter data](#). **Ocean Dynamics**, v. 70, n. 3, p. 405–419, 2020.

FARINA, Leandro. [On ensemble prediction of ocean waves](#). **Tellus A: Dynamic Meteorology and Oceanography**, v. 54, n. 2, p. 148–158, 2002.

HOFFMANN, Matheus. **ww3-gpu: An open lab for WAVEWATCH III operational optimisation and GPU kernels**. <https://github.com/h0ffmann/ww3-gpu>, 2026.

IKUYAJOLU, Olawale James *et al.* [Porting the WAVEWATCH III \(v6.07\) wave action source terms to GPU](#). **Geoscientific Model Development**, v. 16, p. 1445–1458, 2023.

KOLDUNOV, Nikolay V. *et al.* [An ocean model ported by a large language model: Experience and lessons from FESOM2 \(Fortran to C to C++/Kokkos\)](#). arXiv:2606.11356, 2026.

LABECO/UFSC. **ReNOMO** – rede nacional de observação e monitoramento oceânico. <https://labeco.ufsc.br/renomom/>, 2026.

NOAA-EMC. **WAVEWATCH III** source code repository. <https://github.com/NOAA-EMC/WW3>, 2026a.

NOAA-EMC. **WAVEWATCH IV (WW4)** repository. <https://github.com/NOAA-EMC/WW4>, 2026b.

NOAA/NWS/NCEP. **WAVEWATCH IV: Plan for the next-generation wave model**. *[S.l.]*: NCEP, 2025.

NOAA/NWS/NCEP/EMC. **Operational wave models**. https://www.emc.ncep.noaa.gov/emc/pages/numerical_forecast_systems/wavemodels.php, 2026.

THE WAMDI GROUP. **The WAM model—a third generation ocean wave prediction model**. *Journal of Physical Oceanography*, v. 18, n. 12, p. 1775–1810, 1988.

THE WAVEWATCH III DEVELOPMENT GROUP. **User manual and system documentation of WAVEWATCH III version 6.07**. College Park, MD, USA: NOAA/NWS/NCEP/MMAB, 2019.

TOLMAN, Hendrik L. **A third-generation model for wind waves on slowly varying, unsteady, and inhomogeneous depths and currents**. *Journal of Physical Oceanography*, v. 21, n. 6, p. 782–797, 1991.

TOLMAN, Hendrik L. **The WAVEWATCH IV project: Phase II report**. *[S.l.]*: NOAA/NWS/NCEP/EMC, 2026.

TROTT, Christian R. *et al.* **Kokkos 3: Programming model extensions for the exascale era**. *IEEE Transactions on Parallel and Distributed Systems*, v. 33, n. 4, p. 805–817, 2022.

UFSC. **Laboratório de engenharia e ciências oceânicas (LabECO)**. <https://emc.ufsc.br/portal/laboratorios-2/labeco/>, 2026.

YUAN, Ye *et al.* **WAM6-GPU v1.0: GPU-accelerated third-generation wave model**. *Geoscientific Model Development*, v. 17, p. 6123–6136, 2024.

Rio de Janeiro, September 16, 2026.

Matheus Hoffmann Fernandes Santos - Student

To be defined, Dr. - Advisor

Pedro Veras Guimarães, Dr. - Co-advisor